

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010

OPERATING SYSTEM

WEEK 7 LECTURE 1

WRITING LINUX KERNEL MODULE

WRITING LINUX KERNEL MODULE

Kernel modules

- ★ Kernel modules are piece of code, that can be loaded and unloaded from kernel on demand.
- ★ Kernel modules offer an easy way to extend the functionality of the base kernel without having to rebuild or recompile the kernel again.
- ★ Most of the drivers are implemented as Linux kernel modules.
- ★ When those drivers are not needed, we can unload only those specific drivers, which will reduce the kernel image size.
- ★ The kernel modules will have a .ko extension.
- ★ On a normal linux system, the kernel modules will reside inside **`/lib/modules/<kernel_version>/kernel/`** directory.

WRITING LINUX KERNEL MODULE

Utilities to Manipulate Kernel Modules

Ismod – List Modules that Loaded Already

- ★ Ismod command will list modules that are already loaded in the kernel as shown.

\$lsmod

Module	Size	Used by
snd_intel8x0	40960	2
snd_ac97_codec	131072	1 snd_intel8x0
ac97_bus	16384	1 snd_ac97_codec
snd_pcm	106496	2 snd_ac97_codec,snd_intel8x0
snd_seq_midi	16384	0
snd_seq_midi_event	16384	1 snd_seq_midi
snd_rawmidi	32768	1 snd_seq_midi
snd_seq	69632	2
snd_seq_midi_event,snd_seq_midi		
snd_seq_device	16384	3
snd_seq,snd_rawmidi,snd_seq_midi		
snd_timer	32768	2 snd_pcm,snd_seq
crct10dif_pclmul	16384	0
snd	81920	11
snd_ac97_codec,snd_intel8x0,snd_timer,snd_pcm,snd_seq,snd_rawmidi,snd_seq_device		
crc32_pclmul	16384	0
joydev	20480	0
soundcore	16384	1 snd
i2c_piix4	24576	0
aesni_intel	167936	0
aes_x86_64	20480	1 aesni_intel
lrw	16384	1 aesni_intel
gf128mul	16384	1 lrw
glue_helper	16384	1 aesni_intel
ablk_helper	16384	1 aesni_intel
cryptd	20480	2 aesni_intel,ablk_helper
input_leds	16384	0
serio_raw	16384	0
8250_fintek	16384	0
mac_hid	16384	0
parport_pc	32768	0
ppdev	20480	0
lp	20480	0
parport	49152	3 lp,ppdev,parport_pc
autofs4	40960	2
hid_generic	16384	0
usbhid	49152	0
hid	122880	2 hid_generic,usbhid
psmouse	126976	0
fjes	28672	0
ahci	36864	2
e1000	135168	0
libahci	32768	1 ahci
video	40960	0
pata_acpi	16384	0

WRITING LINUX KERNEL MODULE

❏ Utilities to Manipulate Kernel Modules

insmod – Insert Module into Kernel

- ✳ insmod command will insert a new module into the kernel

\$sudo insmod welcomeinha.ko

\$lsmod | grep “welcomeinha”

Module	Size	Used by
welcomeinha	16384	0
snd_intel8x0	40960	2
snd_ac97_codec	131072	1 snd_intel8x0
ac97_bus	16384	1 snd_ac97_codec
snd_pcm	106496	2 snd_ac97_codec,snd_intel8x0
snd_seq_midi	16384	0
snd_seq_midi_event	16384	1 snd_seq_midi
snd_rawmidi	32768	1 snd_seq_midi
snd_seq	69632	2
snd_seq_midi_event,snd_seq_midi		
snd_seq_device	16384	3
snd_seq,snd_rawmidi,snd_seq_midi		
snd_timer	32768	2 snd_pcm,snd_seq
crct10dif_pclmul	16384	0
snd	81920	11
snd_ac97_codec,snd_intel8x0,snd_timer,snd_pcm,snd_seq,snd_rawmidi,snd_seq_device		
crc32_pclmul	16384	0
joydev	20480	0
soundcore	16384	1 snd
i2c_piix4	24576	0
aesni_intel	167936	0
aes_x86_64	20480	1 aesni_intel
lrw	16384	1 aesni_intel
gf128mul	16384	1 lrw
glue_helper	16384	1 aesni_intel
ablk_helper	16384	1 aesni_intel
cryptd	20480	2 aesni_intel,ablk_helper
input_leds	16384	0
serio_raw	16384	0
8250_fintek	16384	0
mac_hid	16384	0
parport_pc	32768	0
ppdev	20480	0
lp	20480	0
parport	49152	3 lp,ppdev,parport_pc
autofs4	40960	2
hid_generic	16384	0
usbhid	49152	0
hid	122880	2 hid_generic,usbhid
psmouse	126976	0
fjes	28672	0
ahci	36864	2
e1000	135168	0
libahci	32768	1 ahci
video	40960	0
pata_acpi	16384	0

WRITING LINUX KERNEL MODULE

❑ Utilities to Manipulate Kernel Modules

modinfo – Display Module Info

- ★ modinfo command will display information about a kernel module as shown below.

\$modinfo welcomeinha.ko

```
filename:      /home/drarn/welcomeinha/welcomeinha.ko
description:   Welcome Inha University Tashkent module
author:        DR_ARN
license:       GPL
srcversion:    D50DEE0E7F21E291FA2FFAF
depends:
vermagic:     4.4.16 SMP mod_unload modversions
```

\$modinfo modpass.ko

```
filename:      /home/drarn/modpass/modpass.ko
author:        DRARN
license:       GPL
srcversion:    3ADD063D8971A6CC6EE271D
depends:
vermagic:     4.4.16 SMP mod_unload modversions
parm:          myshort:A short integer (short)
parm:          myint:An integer (int)
parm:          mylong:A long integer (long)
parm:          mystring:A character string (charp)
parm:          myintArray:An array of integers
                (array of int)
```

❑ Utilities to Manipulate Kernel Modules

rmmod – Remove Module from Kernel

- ★ rmmod command will remove a module from the kernel. You cannot remove a module which is already used by any program.

```
$sudo rmmod welcomeinha.ko
```

❑ Utilities to Manipulate Kernel Modules

modprobe – Add or Remove modules from the kernel

- ★ modprobe is an intelligent command which will load/unload modules based on the dependency between modules.
- ★ modprobe utility is used to add loadable modules to the Linux kernel. You can also view and remove modules using modprobe command.
- ★ **Linux maintains `/lib/modules/$(uname -r)` directory for modules and its configuration files (except `/etc/modprobe.conf` and `/etc/modprobe.d`).**

WRITING LINUX KERNEL MODULE

modprobe – Add or Remove modules from the kernel -
Example:

The following series of commands illustrate a common way to use modprobe. Each command is prefixed with sudo, since they require root permissions:

```
$sudo ln -s /home/yourdirname/modpass/modpass.ko /lib/modules/$(uname -r)
$sudo depmod -a
$sudo modprobe modpass
$sudo modprobe -r modpass
```

These commands perform the following operations:

1. In the first command, we use ln to create a symbolic link to our module file in the directory `/lib/modules/kernel-release`. The command `$(uname -r)` is executed by the shell and translates to the appropriate string representing our kernel release version.
2. In the second command, an updated dependency list is generated by `depmod -a` to make sure that the module we're installing is aware of all existing modules and dependencies. This dependency list will be used by modprobe when installing the module in the third command.
3. The third command `modprobe` installs the kernel module.
4. The fourth command `modprobe -r` uninstalls the kernel module

WRITING LINUX KERNEL MODULE

□ printk

- ★ **printk is a function that prints messages and is used in the C Programming Language exclusively for the Linux Kernel.**
- ★ It accepts a string parameter called the format string, which specifies a method for rendering an arbitrary number of varied data type parameter(s) into a string. The string is then printed to the kernel log.
- ★ It provides a printf-like abstraction and its parsing of the format string and arguments behave exactly the same way.
- ★ It acts as a debugging tool for kernel programmers who need this function for logging messages from the kernel.
- ★ The printk function prototype is:

```
int printk(loglevel format string, args);
```
- ★ **In kernel module, you can't use the standard C library, so printf is not available, hence the need for printk.**

WRITING LINUX KERNEL MODULE

❑ printk Logging Levels

printk has an optional prefix string: Loglevel.

Loglevel specifies the type of message being sent to the kernel message log.

The syntax with loglevel is:

```
printk(KERN_DEBUG "Debug message shown!\n");
```

Different Loglevels, along with their numerical values, are shown here:

0	KERN_EMERG	Emergency condition, system is probably dead
1	KERN_ALERT	Some problem has occurred, immediate attention is needed
2	KERN_CRIT	A critical condition
3	KERN_ERR	An error has occurred
4	KERN_WARNING	A warning
5	KERN_NOTICE	Normal message to take note of
6	KERN_INFO	Some information
7	KERN_DEBUG	Debug information related to the program

When a log level is not specified, the default log level is KERN_WARNING, unless a different default has been set in the kernel itself.

Loglevels are defined in /usr/src/linux-headers-4.4.16/include/linux/kern_levels.h. Which log levels are printed is configured in the /proc/sys/kernel/printk sysctl file (format: console level, default message level, minimum console level, default console level 4 4 1 7)

WRITING LINUX KERNEL MODULE

- ★ Write a Simple welcomeinha Kernel Module

1. Installing the linux headers

- ★ Install the linux-headers using apt-get

\$apt-get install build-essential linux-headers-\$(uname -r)

2. Create a separate directory for every kernel module

\$mkdir welcomeinha

\$cd welcomeinha

\$gedit welcomeinha.c

WRITING LINUX KERNEL MODULE

* Write a Simple WELCOME INHA Kernel Module

welcomeinha Module Source Code

* Next, create the following welcomeinha.c module in C programming language.

```
#include <linux/module.h> // included for all kernel modules
#include <linux/kernel.h> // included for KERN_INFO
#include <linux/init.h> // included for __init and __exit macros

MODULE_LICENSE("GPL");
MODULE_AUTHOR("DR_ARN");
MODULE_DESCRIPTION("Welcome Inha University Tashkent module");
static int __init welcomeinha_init(void)
{
    printk(KERN_INFO "LOADING WELCOME INHA MODULE.....\n\n\n");
    printk(KERN_INFO "*****\n");
    printk(KERN_INFO "* WELCOME TO INHA UNIVERSITY TASHKENT *\n");
    printk(KERN_INFO "* WISH YOU ALL BEST OF LUCK *\n");
    printk(KERN_INFO "*****\n");
    return 0; // Non-zero return means that the module couldn't be loaded.
}

static void __exit welcomeinha_cleanup(void)
{
    printk(KERN_INFO "*****\n");
    printk(KERN_INFO "* GOOD DAY *\n");
    printk(KERN_INFO "* GOOD BYE *\n");
    printk(KERN_INFO "*****\n");
    printk(KERN_INFO "\n\n\nCleaning up module.....\n");
}

module_init(welcomeinha_init);
module_exit(welcomeinha_cleanup);
```

Warning: All kernel modules will operate on kernel space, a highly privileged mode. So be careful with what you write in a kernel module.

WRITING LINUX KERNEL MODULE

❑ Create Makefile to Compile Kernel Module

- ★ The following makefile can be used to compile the **welcomeinha** kernel module.

\$vi Makefile

obj-m += welcomeinha.o

all:

make -C /lib/modules/\$(shell uname -r)/build M=\$(PWD) modules

clean:

make -C /lib/modules/\$(shell uname -r)/build M=\$(PWD) clean

Use the **make command** to compile **welcomeinha** kernel module as shown below.

\$make

\$ls

welcomeinha.ko kernel module is created

WRITING LINUX KERNEL MODULE

❑ Insert the Sample Kernel Module

- ★ Now that we have our welcomeinha.ko file, we can insert this module to the kernel by using insmod command as shown below.

\$sudo insmod welcomeinha.ko

\$dmesg | tail -7

```
[ 7435.391179] LOADING WELCOME INHA MODULE.....
```

```
[ 7435.391181] *****  
[ 7435.391182] *   WELCOME TO INHA UNIVERSITY TASHKENT   *  
[ 7435.391182] *           WISH YOU ALL BEST OF LUCK           *  
[ 7435.391183] *****
```

- ★ **When a module is inserted into the kernel, the module_init macro will be invoked, which will call the function welcomeinha_init.**
- ★ **Using dmesg command, we can see the output from the sample Kernel module.**
- ★ Please note that printk is a function which is defined in kernel, and it behaves similar to the printf in the IO library. Remember that you cannot use any of the library functions from the kernel module
- ★ **Open another terminal by using xterm and in that terminal type**
\$tail -f /var/log/kern.log

WRITING LINUX KERNEL MODULE

```
$sudo insmod welcomeinha.ko
```

```
$lsmod
```

Module	Size	Used by
welcomeinha	16384	0
snd_intel8x0	40960	2
snd_ac97_codec	131072	1 snd_intel8x0
ac97_bus	16384	1 snd_ac97_codec
snd_pcm	106496	2 snd_ac97_codec,snd_intel8x0
snd_seq_midi	16384	0
snd_seq_midi_event	16384	1 snd_seq_midi
snd_rawmidi	32768	1 snd_seq_midi
snd_seq	69632	2
snd_seq_midi_event,snd_seq_midi		
snd_seq_device	16384	3
snd_seq,snd_rawmidi,snd_seq_midi		
snd_timer	32768	2 snd_pcm,snd_seq
crct10dif_pclmul	16384	0
snd	81920	11
snd_ac97_codec,snd_intel8x0,snd_timer,snd_pcm,snd_seq,snd_rawm		
idi,snd_seq_device		
crc32_pclmul	16384	0
joydev	20480	0
soundcore	16384	1 snd
i2c_piix4	24576	0
aesni_intel	167936	0
aes_x86_64	20480	1 aesni_intel
lrw	16384	1 aesni_intel
gfl28mul	16384	1 lrw
glue_helper	16384	1 aesni_intel
ablk_helper	16384	1 aesni_intel
cryptd	20480	2 aesni_intel,ablk_helper
input_leds	16384	0
serio_raw	16384	0
8250_fintek	16384	0
mac_hid	16384	0
parport_pc	32768	0
ppdev	20480	0
lp	20480	0
parport	49152	3 lp,ppdev,parport_pc
autofs4	40960	2
hid_generic	16384	0
usbhid	49152	0
hid	122880	2 hid_generic,usbhid
psmouse	126976	0
fjes	28672	0
ahci	36864	2
e1000	135168	0
libahci	32768	1 ahci
video	40960	0
pata_acpi	16384	0

modinfo – Display Module Info

- ★ modinfo command will display information about a kernel module as shown below.

\$modinfo welcomeinha.ko

```
filename:          /home/drarn/welcomeinha/welcomeinha.ko
description:       Welcome Inha University Tashkent module
author:           DR_ARN
license:          GPL
srcversion:        D50DEE0E7F21E291FA2FFAF
depends:
vermagic:          4.4.16 SMP mod_unload modversions
```

WRITING LINUX KERNEL MODULE

❑ Remove the Sample Kernel Module

- ★ Now that we can remove this module to the kernel by using `rmmod` command as shown below.

\$sudo rmmod welcomeinha.ko

\$dmesg | tail -8

```
[ 8072.542799] *****  
[ 8072.542801] *                GOOD DAY                *  
[ 8072.542801] *                GOOD BYE                 *  
[ 8072.542802] *****  
[ 8072.542803]
```

Cleaning up module.....

- ★ Similarly, when the module is removed with `rmmod`, `module_exit` macro will be invoked, which will call the `welcomeinha_exit`.
- ★ Using `dmesg` command, we can see the output from the sample Kernel module.

WRITING LINUX KERNEL MODULE

❑ Passing Command Line Arguments to a Kernel Module

- * Modules can take command line arguments, but not with the `argc/argv` you might be used to.
- * To allow arguments to be passed to your module, declare the variables that will take the values of the command line arguments as global and then use the **`module_param()`** macro, (defined in `linux/moduleparam.h`) to set the mechanism up.
- * At runtime, `insmod` will fill the variables with any command line arguments that are given, like **`insmod mymodule.ko myvariable=5`**
- * The variable declarations and macros should be placed at the beginning of the module for clarity.
- * **The `module_param()` macro takes 3 arguments: the name of the variable, its type and permissions for the corresponding file in sysfs.**

```
int myint = 3;
```

```
module_param(myint, int, 0);
```

```
module_param(myint, int, S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
```

- * Integer types can be signed as usual or unsigned.

WRITING LINUX KERNEL MODULE

□ Passing Command Line Arguments to a Kernel Module

- * To use arrays of integers or strings use `module_param_array()` and `module_param_string()`.
- * To keep track of the number of parameters you need to pass, a pointer to a count variable as third parameter. At your option, you could also ignore the count and pass NULL instead.

We show both possibilities here:

```
int myintarray[2];
module_param_array(myintarray, int, NULL, 0); /* not interested in count */
int myshortarray[4];
int count;
module_param_array(myshortarray, short, &count, 0); /* put count into "count" variable */
```

- * A good use for this is to have the module variable's default values set, like an port or IO address.
- * If the variables contain the default values, then perform autodetection. Otherwise, keep the current value.
- * Lastly, there's a macro function, `MODULE_PARM_DESC()`, that is used to document arguments that the module can take.
- * It takes two parameters: a variable name and a free form string describing that variable.

`MODULE_PARM_DESC(name, string )`

```
module_param(mystring, charp, 0000);
```

```
MODULE_PARM_DESC(mystring, "A character string");
```

WRITING LINUX KERNEL MODULE

❑ Passing Command Line Arguments to a Kernel Module-EXAMPLE

```
$mkdir modpass && cd modpass && gedit modpass.c
```

```
/* modpass.c- Demonstrates command line argument passing to a module */
```

```
#include <linux/module.h>
```

```
#include <linux/moduleparam.h>
```

```
#include <linux/kernel.h>
```

```
#include <linux/init.h>
```

```
#include <linux/stat.h>
```

```
MODULE_LICENSE("GPL");
```

```
MODULE_AUTHOR("DRARN");
```

```
static short int myshort = 1;
```

```
static int myint = 2024;
```

```
static long int mylong = 999999;
```

```
static char *mystring = "hello";
```

```
static int myintArray[2] = { 16, 10 };
```

```
static int arr_argc = 0;
```

```
/* module_param(name, int, 0000)
```

```
* The first param is the parameters name
```

```
* The second param is it's data type
```

```
* The final argument is the permissions bits,
```

```
*/
```

WRITING LINUX KERNEL MODULE

❑ Passing Command Line Arguments to a Kernel Module-EXAMPLE

```
module_param(myshort, short, S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP);
```

```
MODULE_PARM_DESC(myshort, "A short integer");
```

```
module_param(myint, int, S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
```

```
MODULE_PARM_DESC(myint, "An integer");
```

```
module_param(mylong, long, S_IRUSR);
```

```
MODULE_PARM_DESC(mylong, "A long integer");
```

```
module_param(mystring, charp, 0600);
```

```
MODULE_PARM_DESC(mystring, "A character string");
```

```
/* module_param_array(name, type, num, perm);
```

*The first param is the parameter's (in this case the array's) name

*The second param is the data type of the elements of the array

*The third argument is a pointer to the variable that will store the number

* of elements of the array initialized by the user at module loading time

*The fourth argument is the permission bits */

```
module_param_array(myintArray, int, &arr_argc, 0644);
```

```
MODULE_PARM_DESC(myintArray, "An array of integers");
```

WRITING LINUX KERNEL MODULE

❑ Passing Command Line Arguments to a Kernel Module-EXAMPLE

```
static int __init modpass_init(void)
{
    int i;
    printk(KERN_INFO "Hello, world with Passing command line arguments\n=====\\n");
    printk(KERN_INFO "myshort is a short integer: %hd\\n", myshort);
    printk(KERN_INFO "myint is an integer: %d\\n", myint);
    printk(KERN_INFO "mylong is a long integer: %ld\\n", mylong);
    printk(KERN_INFO "mystring is a string: %s\\n", mystring);
    for (i = 0; i < (sizeof myintArray / sizeof (int)); i++)
    {
        printk(KERN_INFO "myintArray[%d] = %d\\n", i, myintArray[i]);
    }
    printk(KERN_INFO "got %d arguments for myintArray.\\n", arr_argc);
    return 0;
}

static void __exit modpass_exit(void)
{
    printk(KERN_INFO "Goodbye, Hello world passing command line arguments exiting.... \\n");
}

module_init(modpass_init);
module_exit(modpass_exit);
```

WRITING LINUX KERNEL MODULE

❑ Create Makefile to Compile Kernel Module

- * The following makefile can be used to compile the kernel module modpass.

\$vi Makefile

obj-m +=modpass.o

all:

make -C /lib/modules/\$(shell uname -r)/build M=\$(PWD) modules

clean:

make -C /lib/modules/\$(shell uname -r)/build M=\$(PWD) clean

Use the make command to compile **modpass** kernel module as shown below.

\$make

\$ls **modpass.ko** **kernel module is created**

**\$sudo insmod modpass.ko myshort=6 myint=100 mylong=29999
mystring="GREAT" myintArray=2,4**

WRITING LINUX KERNEL MODULE

\$dmesg | tail -9 > modinit

\$cat modint

```
[11441.687463] Hello, world  with Passing command line arguments
                =====
[11441.687465] myshort is a short integer: 6
[11441.687466] myint is an integer: 100
[11441.687467] mylong is a long integer: 299999
[11441.687468] mystring is a string: GREAT
[11441.687468] myintArray[0] = 2
[11441.687469] myintArray[1] = 4
[11441.687470] got 2 arguments for myintArray.
```

\$sudo rmmod modpass.ko

\$ dmesg | tail -1 > modexit

\$cat modexit

```
11556.862476] Goodbye, Hello world  passing command
line arguments exiting ...
```

modinfo – Display Module Info

- ★ modinfo command will display information about a kernel module as shown below.

\$modinfo modpass.ko

```
$modinfo modpass.ko
filename:          /home/drarn/modpass/modpass.ko
author:           DRARN
license:          GPL
srcversion:       3ADD063D8971A6CC6EE271D
depends:
vermagic:         4.4.16 SMP mod_unload modversions
parm:             myshort:A short integer (short)
parm:             myint:An integer (int)
parm:             mylong:A long integer (long)
parm:             mystring:A character string (charp)
parm:             myintArray:An array of integers (array of int)
```

INTERRUPTS & EXCEPTIONS

❑ Questions

- * Explain what is meant by a kernel module. Why is it required?
- * Describe the following utilities used to manipulate Kernel modules. Explain how to use these commands taking an example.
 - a) `lsmod`
 - b) `insmod`
 - c) `modinfo`
 - d) `rmmod`
 - e) `modprobe` .
- * What is `printk`? Write the syntax of `printk` function.
- * Describe the `printk` logging levels.
- * Explain what happens when the following commands are executed :
 - a) `dmesg`
 - b) `dmesg | tail -10`
 - c) `tail -f /var/log/kern.log`
- * Describe the purpose of the following macros :
 - a) `module_init()` b) `module_exit()` c) `MODULE_PARAM()`
 - d) `MODULE_PARM_DESC()` e) `MODULE_LICENSE()`
 - f) `MODULE_AUTHOR()` g) `MODULE_DESCRIPTION()`
- * Write the Makefile used to compile and build the kernel module `welcomeinha.ko`
- * Say TRUE OR FALSE - In Kernel module, you can use the Standard C library functions

SOC 3010

OPERATING SYSTEM

**END OF
WEEK 7 LECTURE 1**

**WRITING LINUX
KERNEL MODULE**

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG